

Stack Implementation in C++ (With Comments)

Course Notes for Students

1. Introduction to Stack

A Stack is a linear data structure that follows the LIFO principle (Last In, First Out). Insertion and deletion operations are performed only at one end, called the top.

2. Basic Stack Operations

- Push: Insert an element onto the stack
- Pop: Remove the top element from the stack
- Peek: View the top element without removing it
- isEmpty: Check if the stack is empty
- isFull: Check if the stack is full

3. Stack Using Array

```
class Stack {
    int arr[100];
    int top;

public:
    Stack() {
        top = -1;
    }

    bool isEmpty() {
        return (top == -1);
    }

    bool isFull() {
        return (top == 99);
    }

    void push(int value) {
        if (isFull()) return;
        arr[++top] = value;
    }

    void pop() {
        if (isEmpty()) return;
        top--;
    }

    int peek() {
        if (isEmpty()) return -1;
        return arr[top];
    }
};
```

4. Stack Using Linked List

```
struct Node {
    int data;
    Node* next;
};

class Stack {
```

```

    Node* top;

public:
    Stack() {
        top = nullptr;
    }

    bool isEmpty() {
        return (top == nullptr);
    }

    void push(int value) {
        Node* temp = new Node();
        temp->data = value;
        temp->next = top;
        top = temp;
    }

    void pop() {
        if (isEmpty()) return;
        Node* temp = top;
        top = top->next;
        delete temp;
    }

    int peek() {
        if (isEmpty()) return -1;
        return top->data;
    }
};

```

5. Applications of Stack

- Function calls and recursion
- Expression evaluation (postfix/prefix)
- Undo and redo operations
- Parenthesis checking

6. Summary

- Stack follows LIFO principle
- Array-based stack has fixed size
- Linked list stack allows dynamic size
- Stack operations are efficient ($O(1)$)